

ROI Dashboard Requirements

1) Goal / What success looks like

Build a Client Solutions ROI & Adoption Dashboard that:

1. Gives a portfolio view of every client + solution (usage, savings, ROI status).
2. Lets an admin (Margaux) add/edit projects with dynamic input fields (hourly rate, cost, launch date, etc.).
3. Tracks usage trends over time (30d, 3mo, 12mo + MoM deltas, since launch).
4. Triggers an alert email to Margaux's inbox when adoption/usage drops or becomes abnormal

2) Dashboard Views (UI requirements)

2.1 Portfolio Overview

A view where each row = solution, grouped by client, with:

- Client, Solution, Active?
- Usage: last month (30d), MoM %, trailing 3mo avg, YTD/12mo
- Savings: last month \$, MoM \$, trailing 3mo avg \$, cumulative saved
- ROI: monthly target, last month ROI value, ROI status badge, ROI reached?, break-even estimate

Table features:

- Filter: by client, active/inactive, ROI status, "needs attention"
- Sort by: biggest drop, lowest usage, highest savings, closest to ROI, etc.
- Drilldown click on a solution -> Solution Detail

2.2 Client Detail

- List of all solutions for that client
- Aggregate totals for client (total savings, total investment, portfolio ROI net)
- Trend chart(s): usage + savings over time

2.3 Solution Detail

- Config panel (editable): hourly rate, project cost, launch date, time saved per usage, monthly target, active toggle
- Usage trend chart (daily/weekly/monthly)
- Savings trend chart
- ROI progress bar + ROI net number
- Alerts history (what triggered, when, acknowledged)

2.4 Admin: Add / Edit Solution

Form to create a new tracked solution with required fields:

- client, solution name, launch date, project cost, hourly rate, time saved per usage minutes, usage unit label
- Optional:
- monthly ROI target, source link, active toggle

Core Entities (Data Model)

3.1 Client

- `client_id`
- `client_name`
- optional: `client_owner` (internal owner), `notes`

3.2 Solution / Project (the unit you track ROI for)

One row = one deployed solution for a client (ex: Cellcom – HR Chatbot).

Fields:

- Identifiers:
 - `solution_id`
 - `client_id`
 - `solution_name`
 - `is_active`
- Investment & dates:

- `project_cost` (investment, \$)
 - `launch_date` (go-live)
- Savings configuration (editable):
 - `hourly_rate` (\$/hour) **default 45** but editable per solution
 - `time_saved_per_usage_minutes` (minutes saved per usage/action)
 - `usage_unit_label` (ex: "Questions Answered", "Workflow Ran", "Reports Exported")
- Targets / status:
 - `monthly_roi_target` (\$/month) (optional but used for "On track")
 - `roi_status_ruleset` (optional: per-solution thresholds overrides)
- References:
 - `source_link` (link to sheet/log/source system)

2.3 Usage Events (raw time series)

We need to store usage as a **time series** so we can compute trailing windows and drop alerts.

Fields:

- `usage_event_id`
- `solution_id`
- `event_date` (date, in America/Montreal)
- `usage_count` (integer >= 0)
- optional:
 - `usage_type` (if a solution has multiple actions)
 - `source` (manual / API / import)

Important: some sources are daily counts (ex "HQ - Hemy"), others might be monthly totals. Support both; normalize by storing `event_date` + `usage_count`.

3) Computed Metrics

The dashboard must compute these **per solution**:

3.1 Usage metrics

- `usage_last_30_days`
- `usage_last_3_months`
- `usage_last_12_months`
- `mom_usage_percent` (Month-over-month % change)
- `trailing_3mo_monthly_avg_usage` (avg monthly usage across last 3 months)
- `usage_ytd` (optional if you want a YTD view)

3.2 Time saved

From usage and time saved per usage:

- `time_saved_hours_30d = usage_last_30_days * time_saved_per_usage_minutes / 60`
- `time_saved_hours_3mo = usage_last_3_months * time_saved_per_usage_minutes / 60`
- `time_saved_hours_12mo = usage_last_12_months * time_saved_per_usage_minutes / 60`

3.3 Cost saved (\$)

- `cost_saved_30d = time_saved_hours_30d * hourly_rate`
- `cost_saved_3mo = time_saved_hours_3mo * hourly_rate`
- `cost_saved_12mo = time_saved_hours_12mo * hourly_rate`
- `cumulative_cost_saved_since_launch` (sum from `launch_date` to today; if you only have recent data, sum what exists and show “partial coverage” flag)

3.4 ROI tracking

Define ROI as:

- `roi_net = cumulative_cost_saved_since_launch - project_cost`

Also compute:

- `roi_reached` (boolean): `roi_net >= 0`
- `roi_progress_percent` = `cumulative_cost_saved_since_launch / project_cost` (cap at >100% allowed)
- `roi_status_badge` (example):
 - Green: above target
 - Yellow: on track
 - Red: below target
 - Gray: inactive / no data

3.5 “When will ROI be reached?” estimate

If ROI not reached, estimate break-even date using recent monthly savings:

- `recent_monthly_savings` = `cost_saved_3mo / 3` (or last 30d * 30/30 if you prefer)
- `months_to_break_even` = `(project_cost - cumulative_cost_saved_since_launch) / max(recent_monthly_savings, epsilon)`
- `estimated_break_even_date` = `today + months_to_break_even`

If `recent_monthly_savings` ~ 0, show “Cannot estimate (insufficient usage)”.

Ensure to keep all historical data (i.e. I need to easily be able to access the 2025 monthly data for a solution)

3) Alerts (email to inbox)

3.1 Alert types (minimum)

A) Usage Drop Alert

Trigger when any of these conditions are met (configurable globally + per-solution override). Give me the ability to manually set personalised alerts per client. These are the kinds of alerts I would set:

- `usage_last_30_days` is down by **X% vs previous 30 days** (default X=30%)
- OR `usage_last_30_days` is below **Y% of trailing 3-month monthly average** (default Y=60%)
- OR **0 usage in last N days** for an active solution (default N=14)

B) ROI Off-track Alert (optional but valuable)

Trigger if:

- `cost_saved_30d < monthly_roi_target * threshold` (default threshold = 0.8)

5.2 Alert behavior

- Send an **email** to a configured address (my inbox) containing:
 - client + solution
 - what triggered (metric values + threshold)
 - link to solution detail page
- Deduping:
 - Don't spam: once triggered, don't resend the same alert for **7 days** unless it worsens by a lot (configurable).
- Alert states:
 - Have a little notifications section where I can login into the portal and see all past and current alerts. Must be able to filter by client and solution.